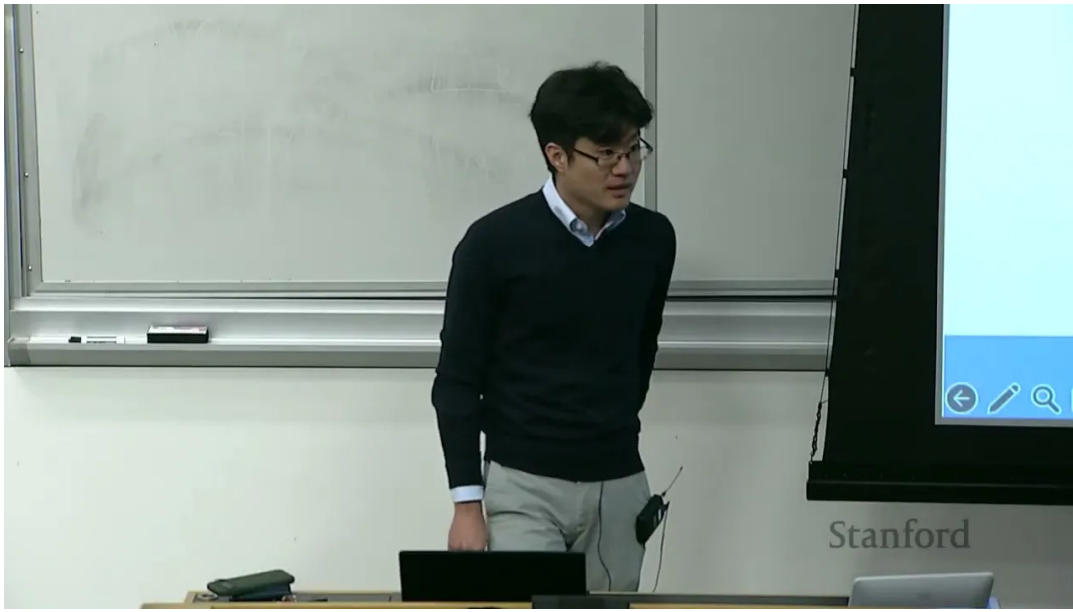


课程讲义

GPU 性能的第一性原理

Stanford CS336 Spring 2025 · Lecture 5 · GPUs



频道：Stanford Online

发布日期：2025 年 5 月 1 日

视频时长：1 小时 14 分 21 秒

视频链接：<https://www.youtube.com/watch?v=60Bt09niT00>

整理说明：依据完整英文字幕、原始全帧课件与逐图时间窗核验重构。重点不是记住 GPU 名词，而是建立能预测 kernel 快慢的计算—带宽—调度模型。

整理日期：2026 年 7 月 29 日

目录

1	为什么语言模型离不开 GPU	3
1.1	从缩放需求到并行机器	3
1.2	本章小结	5
2	GPU 的硬件与执行模型	6
2.1	线程多而轻，控制少而共享	6
2.2	本章小结	8
3	存储层级与 Roofline	9
3.1	数据搬运决定上限	9
3.2	本章小结	11
4	分支、精度与算术强度	12
4.1	SIMT 下的控制流分歧	12
4.2	低精度同时减少字节并启用专用单元	13
4.3	本章小结	14
5	算子融合与重计算	14
5.1	把“仓库往返”变成“工厂内流水”	14
5.2	重计算：用便宜 FLOPs 换昂贵访存	15
5.3	本章小结	16
6	合并访问、分块与对齐	16
6.1	让相邻线程访问相邻地址	16
6.2	分块提升复用	18
6.3	本章小结	19
7	调度粒度与波量化	20
7.1	矩阵只大一个元素为何可能突然变慢	20
7.2	本章小结	21
8	FlashAttention：把原则组合成算法	21
8.1	普通注意力的中间矩阵	21
8.2	在线 softmax 保持数值稳定	22
8.3	本章小结	24
9	性能诊断推演与复习	25

9.1	从症状建立假设	25
9.2	建立形状矩阵	25
9.3	优化阶梯	25
9.4	FlashAttention 的迁移练习	26
9.5	复习问题	26
9.6	本章小结	26

1 为什么语言模型离不开 GPU

1.1 从缩放需求到并行机器

语言模型训练把大量规则的矩阵乘、逐元素变换和归约重复数万亿次。单核延迟不是主要目标，单位时间完成多少样本、多少 token 才是系统指标。CPU 用复杂控制、缓存和乱序执行降低单线程延迟；GPU 则把晶体管投入大量算术单元，用成千上万个并发线程掩盖长延迟。课程首先把问题拆成三层：硬件与执行模型、通用性能技巧、FlashAttention 案例。

Organization today:

❖ **Part 1:** GPUs in depth – how they work and important parts

❖ **Part 2:** Understanding GPU performance

❖ **Part 3:** Putting it together – unpacking FlashAttention

Stanford



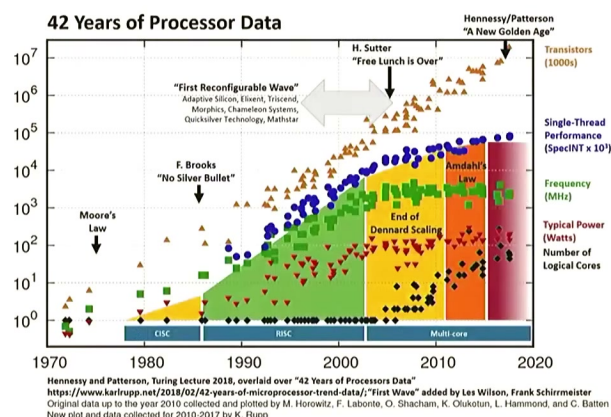
图 1: 本讲从硬件、性能模型进入 FlashAttention。
视频证据时间窗：00:02:55–00:03:35。

若训练总工作量为 W FLOPs、持续有效吞吐为 R_{eff} FLOP/s，则纯计算时间下界为

$$T_{\text{train}} \geq \frac{W}{R_{\text{eff}}}.$$

- W : 模型、数据量与训练步数共同决定的总浮点工作量；
- R_{eff} : 扣除访存、通信、空泡和调度后真正持续的吞吐；
- 不等号提醒我们：峰值算力只能给出乐观下界。

How do we get compute scaling? Early on – Dennard scaing

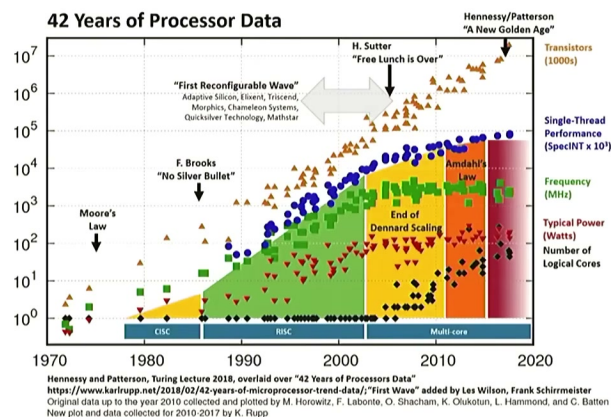


But the traditional form of scaling (*Dennard scaling*) from 1980-2000s has tapped out.
 .. How do we feed LLMs' insatiable appetite for compute?



图 2: 训练需求和硬件算力共同扩展，系统效率决定两者能否接上。
 视频证据时间窗：00:03:55–00:04:35。

How do we get compute scaling? Early on – Dennard scaing



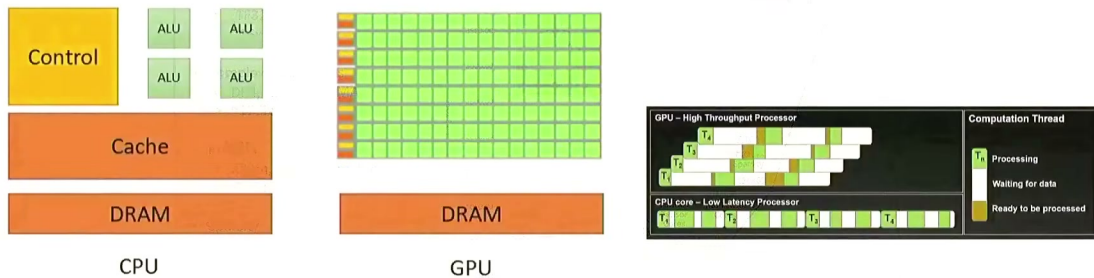
But the traditional form of scaling (*Dennard scaling*) from 1980-2000s has tapped out.
 .. How do we feed LLMs' insatiable appetite for compute?



图 3: Dennard 缩放结束后，频率不再免费增长，并行化成为主要路径。
 视频证据时间窗：00:04:55–00:05:35。

How is a GPU different from a CPU?

CPU's optimize for a few, fast threads while GPUs optimize for many many threads



Many tiny compute units (ALUs).
Much less support for branching (control, cache)

CPU's optimize for latency (each thread finishes quickly)
GPUs optimize for throughput (total processed data)



<https://developer.nvidia.com/blog/cuda-refresher-reviewing-the-origins-of-gpu-computing/>

图 4: 课件以十年约千倍的 GPU 并行算力增长说明系统趋势。

视频证据时间窗: 00:05:55-00:06:35。

峰值 FLOPs 不是训练速度

型号表中的峰值通常假定指定精度、Tensor Core、理想指令混合和满占用。真实训练还要读取权重与激活、执行非矩阵算子、同步并通信。因此比较系统时必须同时报告有效吞吐、模型 FLOPs 利用率和端到端步时。

强缩放效率可写成

$$\eta_p = \frac{T_1}{pT_p}$$

- T_1 : 一张设备完成固定工作量的时间;
- T_p : p 张设备完成同一工作量的时间;
- $\eta_p = 1$ 才是理想线性加速, 实际会因通信和小 kernel 下降。

1.2 本章小结

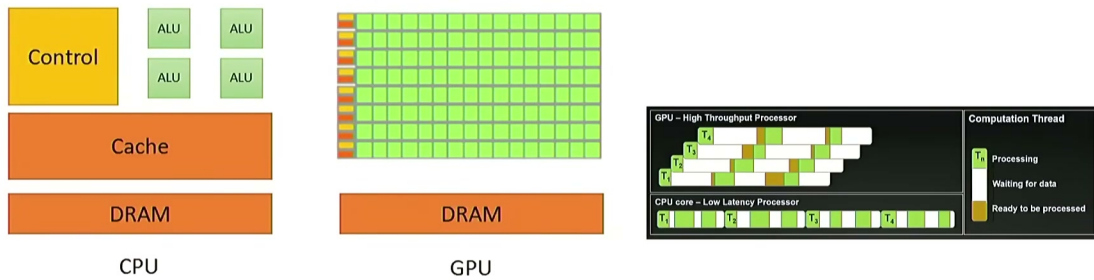
GPU 的价值来自高吞吐并行, 不来自神秘指令。预测训练速度要从总工作量、持续吞吐和并行效率出发; 后续所有技巧都在提高 R_{eff} 或降低不得不搬运的数据量。

2 GPU 的硬件与执行模型

2.1 线程多而轻，控制少而共享

How is a GPU different from a CPU?

CPU's optimize for a few, fast threads while GPUs optimize for many many threads



Many tiny compute units (ALUs).
Much less support for branching (control, cache)

CPU's optimize for latency (each thread finishes quickly)
GPUs optimize for throughput (total processed data)

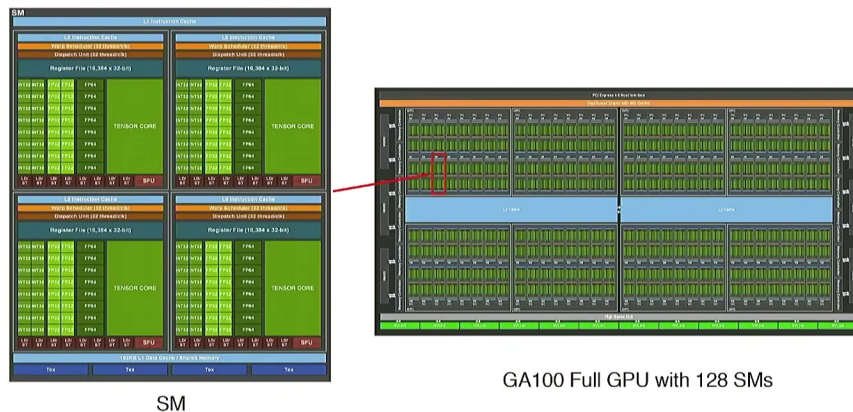


<https://developer.nvidia.com/blog/cuda-refresher-reviewing-the-origins-of-gpu-computing/>

图 5: CPU 追求少量强线程，GPU 追求大量轻量线程。

视频证据时间窗：00:07:10–00:07:50。

Anatomy of a GPU (execution units)



SM

GA100 Full GPU with 128 SMs

Each SM further contains many **SPs (streaming processor)** that can execute 'threads' in parallel

GPUs have many **SM (streaming multiprocessors)** that independently execute 'blocks' (jobs).



图 6: GA100/A100 由大量 Streaming Multiprocessor 组成。

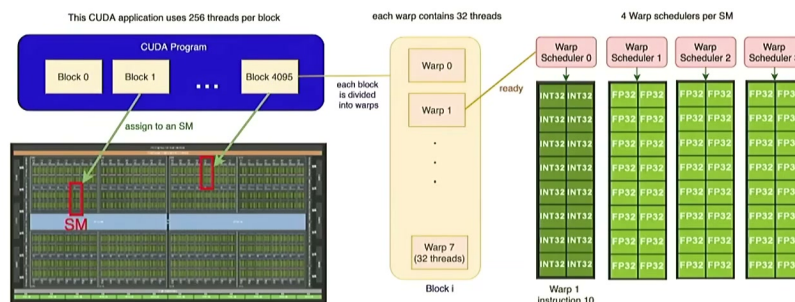
视频证据时间窗：00:09:25–00:10:05。

CUDA 把一次 kernel 启动描述为网格，网格包含线程块，线程块再包含线程。硬件把同一线程块放到一个 SM 上，并把线程按 warp 成组执行。若每块有 B 个线程、网格有 G 个块，则逻辑线程总数为

$$N_{\text{threads}} = GB.$$

- G : 可独立调度的 thread block 数;
- B : 每个 block 内协作的线程数;
- 逻辑线程数可以远大于物理执行单元，硬件分波次推进。

Execution model of a GPU



There are 3 important players in the execution model

Threads: Threads 'do the work' in parallel – all threads execute the same instructions but with different inputs (SIMT).

Blocks: Blocks are groups of threads. Each block runs on a SM w/ its own shared memory.

Warp: Threads always execute in a 'warp' of 32 consecutively numbered threads each.

图 7: grid、thread block 与 thread 的层次映射。

视频证据时间窗: 00:12:55–00:13:35。

为什么需要 thread block

块是调度和局部协作的边界。块内线程可通过 shared memory 和 barrier 交换数据；块与块之间通常不能在同一 kernel 内做全局同步。设计 block 尺寸，就是在并行度、寄存器压力、共享内存和分支行为之间折中。

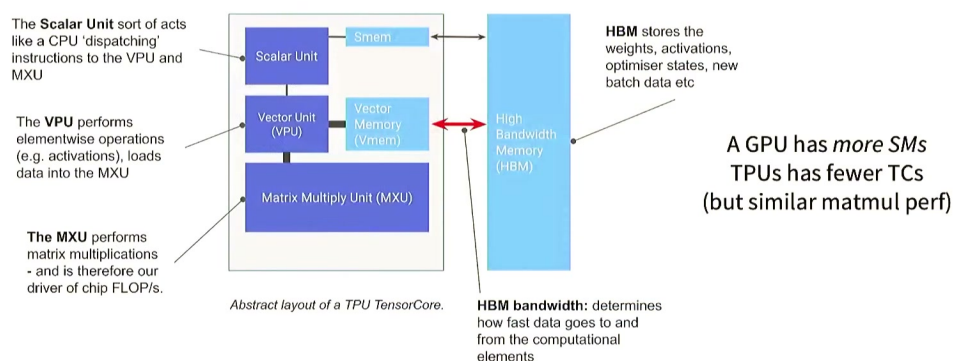
若一个 SM 最多同时驻留 W_{max} 个 warp，实际驻留 W_{active} 个 warp，占用率为

$$\text{occupancy} = \frac{W_{\text{active}}}{W_{\text{max}}}.$$

- W_{active} : 受寄存器、共享内存和线程数共同限制;
- W_{max} : 具体 GPU 架构规定的最大驻留 warp;
- 高占用率有助于隐藏延迟，但不保证 kernel 一定更快。

Side thread – What about TPUs?

GPUs, TPUs, and many other accelerators are at a high level, similar



Core structure – lightweight control, fast (big) matmul unit, fast memory.

Differences - how the accelerators are networked (in the parallelism lecture)
 - no warps (just blocks – tradeoffs in matmul vs non-matmul)

图 8: TPU 与 GPU 都服务高吞吐张量计算, 但执行和存储抽象不同。

视频证据时间窗: 00:16:10–00:16:50。

2.2 本章小结

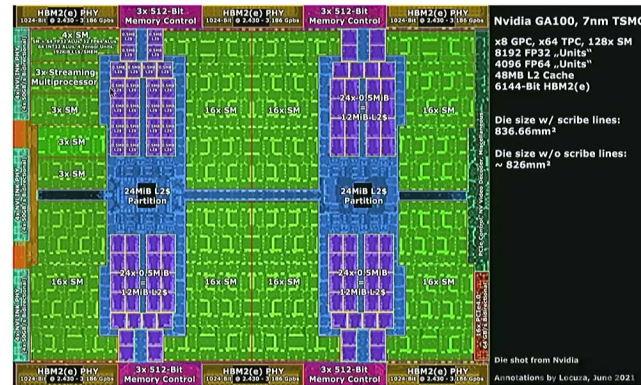
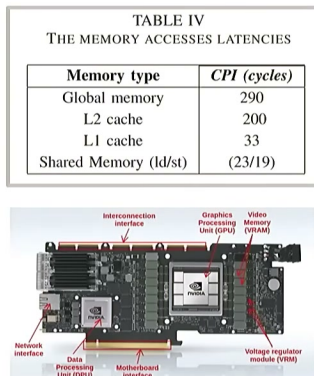
CUDA 的 grid/block/thread 是软件视图, SM/warp 是硬件执行视图。优化 kernel 的第一步是知道每一级资源由谁共享、何时同步, 以及逻辑并行怎样分波映射到有限的物理单元。

3 存储层级与 Roofline

3.1 数据搬运决定上限

Anatomy of a GPU (memory)

The closer the memory to the SM, the faster it is – L1 and shared memory is *inside* the SM. L2 cache is on die, and global memory are the memory chips next to the GPU



SRAM (shared/cache memory) is much more expensive (100x) but ~ 8x faster than DRAM (Global memory)

图 9: 寄存器、片上缓存/共享内存与全局显存的容量—延迟层级。

视频证据时间窗: 00:11:25–00:12:05。

从 HBM 读入一个数往往比对它做一次加法昂贵得多。算术强度把算法需要的计算量除以离开主存的数据量:

$$I = \frac{F}{Q} \quad [\text{FLOP/byte}].$$

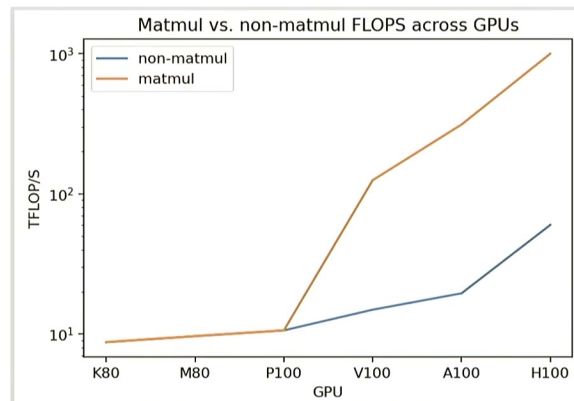
- F : kernel 完成的浮点操作数;
- Q : HBM 与芯片之间实际传输的字节数;
- I 越高, 越可能把瓶颈从带宽推向计算。

Roofline 给出的吞吐上界是

$$P \leq \min(P_{\text{peak}}, I B_{\text{mem}}).$$

- P_{peak} : 目标精度下峰值算力;
- B_{mem} : 可持续显存带宽;
- $I B_{\text{mem}}$: 由数据供给速度允许的吞吐上界。

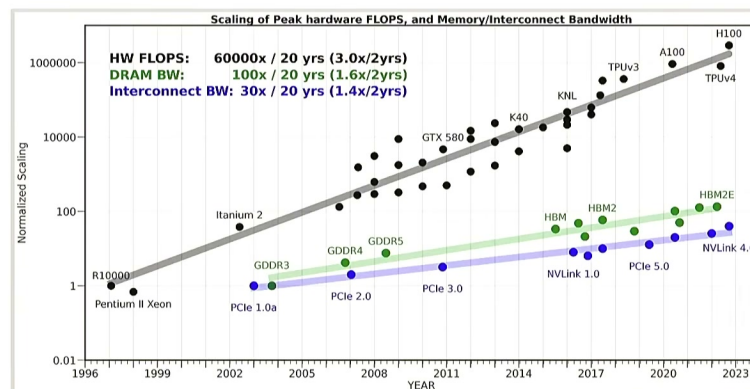
New matmul hardware means matmuls are fast and special



Tensor cores (introduced in V, T series) are specialized matrix multiplication circuits.
Matmuls are >10x faster than other floating point ops!

图 10: Tensor Core 让低精度矩阵乘远快于普通标量浮点操作。
 视频证据时间窗: 00:20:40–00:21:20。

Compute scaling is faster than memory scaling



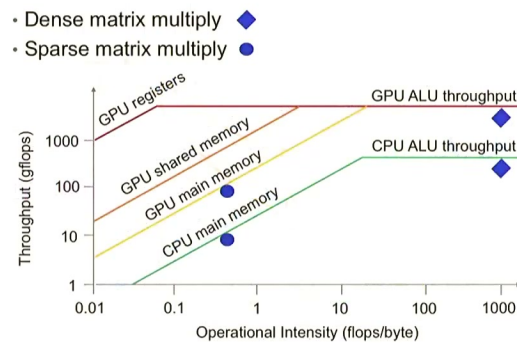
<https://medium.com/riselab/ai-and-memory-wall-2cb4265cb0b8>

FLOPs scale faster than memory – it's hard to keep our compute units fed with data!

图 11: 计算峰值增长快于显存带宽，数据移动越来越关键。
 视频证据时间窗: 00:21:55–00:22:35。

What makes ML workloads fast?

The roofline model



Key to this section: **how do we avoid being memory bound?**

Stanford

图 12: Roofline 将计算受限区和带宽受限区统一到一个图上。

视频证据时间窗: 00:25:55–00:26:35。

算术强度必须按真实流量计算

不能只数源代码中的张量大小。缓存命中、重复加载、写回、临时张量和融合都会改变 Q 。Profiler 的 DRAM bytes 与 kernel 时间，才是判断优化是否奏效的可靠依据。

矩阵乘 $C_{m \times n} = A_{m \times k} B_{k \times n}$ 的理想计算量约为

$$F_{\text{matmul}} \approx 2mnk.$$

- m, n, k : 矩阵三个维度;
- 系数 2: 一次乘法加一次加法;
- 当矩阵足够大且数据被重复利用时, 算术强度远高于逐元素算子。

3.2 本章小结

Roofline 是贯穿全讲的判断框架: 先算强度, 再看位于带宽斜线还是计算平台。矩阵乘擅长复用数据, 逐元素链条容易被中间张量的读写拖住。

4 分支、精度与算术强度

4.1 SIMT 下的控制流分歧

How do we make GPUs go fast?

1. Control divergence (not a memory bottleneck..)
2. Low precision computation
3. Operator fusion
4. Recomputation
5. Coalescing memory
6. Tiling

Stanford

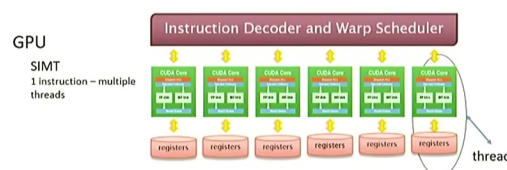


图 13: 课程总结的六类技巧：分歧、低精度、融合、重计算、合并访问和分块。

视频证据时间窗：00:27:10–00:27:50。

Control divergence (not a memory issue)

GPUs operate in a SIMT model – every thread in a warp is executing the same instruction



Conditionals are fine, but lead to significant overhead from the execution model

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```

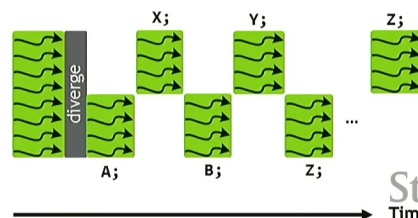


图 14: 同一 warp 走不同分支时，分支往往需要串行执行。

视频证据时间窗：00:27:55–00:28:35。

若一个 warp 的分支路径集合为 $\mathcal{P}$ ，每条路径耗时 t_j ，分歧后的近似时间为

$$T_{\text{warp}} \approx \sum_{j \in \mathcal{P}} t_j.$$

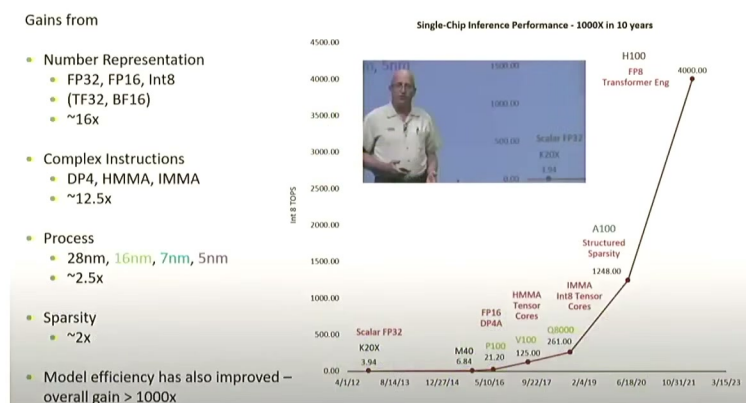
- $\mathcal{P}$: 该 warp 中至少有一个线程进入的路径;
- t_j : 执行路径 j 所需时间;
- 同一 warp 的线程不是独立标量核，分歧可能把并行变成串行。

分支优化的边界

不要机械删除所有条件判断。若一个 warp 内大多数线程走同一路径，分支成本可能很低；若为消除分支而增加大量无用计算，也可能更慢。真正要减少的是 warp 内不一致，而非源代码里的 `if` 数量。

4.2 低精度同时减少字节并启用专用单元

Trick 1: Low precision computation



If you have fewer bits, you have fewer bits to move

Stanford

图 15: 低精度减少存储和带宽需求，并可进入 Tensor Core 快路径。

视频证据时间窗：00:29:10–00:29:50。

对只读一个输入并写一个输出的逐元素函数，若每个元素计算一次 FLOP，则 FP32 的算术强度近似

$$I_{\text{FP32}} = \frac{1}{4+4} = \frac{1}{8} \text{ FLOP/byte.}$$

- 两个 4 字节分别来自一次读与一次写;
- 强度仅 0.125，通常明显带宽受限;
- 中间张量越多，实际强度还会下降。

FP16/BF16 的相同模式为

$$I_{\text{FP16}} = \frac{1}{2+2} = \frac{1}{4} \text{ FLOP/byte.}$$

- 每个元素读写各 2 字节；
- 不改变算法 FLOPs 就把强度翻倍；
- 数值稳定性仍需损失缩放、FP32 累加或更稳健的 BF16。

Low precision improves arithmetic intensity

Example: elementwise ReLU ($x = \max(0, x)$) on a vector of size n .

(Float 32 case)

Memory access: 1 read (x), 1 write (if $x < 0$), float 32 = 8 bytes

Operations: 1 comparison op, 1 FLOP.

Intensity: 8 bytes / FLOP

(Float 16)

Memory access: 1 read (x), 1 write (if $x < 0$), float 16 = 4 bytes

Operations: 1 comparison op, 1 FLOP.

Intensity: 4 bytes / FLOP

Stanford

图 16: 同一逐元素算子在 FP32 与 FP16 下的字节/FLOP 对照。

视频证据时间窗: 00:30:25–00:31:05。

精度选择是一项系统设计

低精度不是简单“少两字节”。数据类型影响内存、通信、Tensor Core 指令、归约误差和优化器状态。训练系统常让权重、激活、梯度、累加器使用不同精度，从而在速度与稳定性之间取得平衡。

4.3 本章小结

控制流分歧降低 SIMT 利用率，低精度同时改善带宽和计算通道。两者都必须用数据验证：看 warp 执行效率、DRAM 流量、Tensor Core 利用率与最终数值误差。

5 算子融合与重计算

5.1 把“仓库往返”变成“工厂内流水”

一串逐元素算子若分别启动 kernel，每一步都把结果写回 HBM，下一步又读回来。融合将中间值保留在寄存器或片上存储，一次读入、一次写出。对 k 个逐元素阶段、每个元素 s 字节，未融合流

量近似

$$Q_{\text{unfused}} \approx 2ksN.$$

- k : 独立 kernel 数;
- s : 每个元素字节数;
- N : 元素个数;
- 每阶段一次读、一次写, kernel launch 还会增加固定开销。

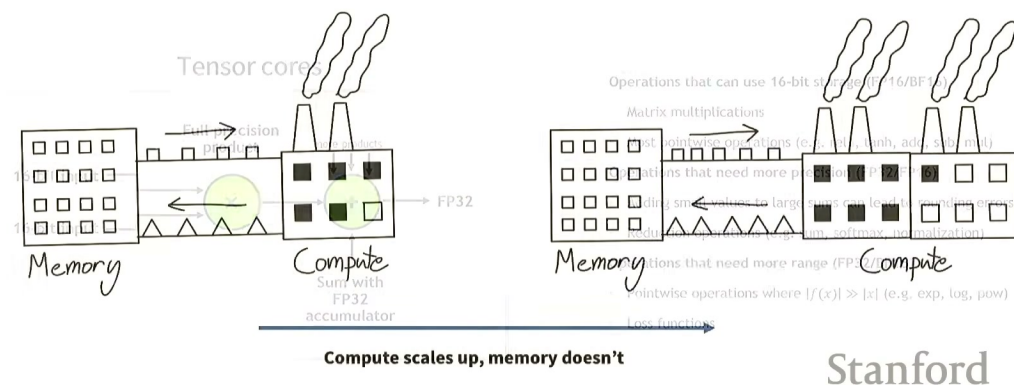
理想融合后的流量为

$$Q_{\text{fused}} \approx 2sN.$$

- 只从全局内存读一次输入、写一次结果;
- 理想流量收益约为 k 倍;
- 寄存器溢出、复杂归约和形状不兼容会限制融合。

Trick 2: Operator fusion

Think of a GPU like a factory – inputs come from a warehouse (memory) and is processed at a factory



Compute scales up, memory doesn't

Stanford

图 17: 多个逐元素节点融合成单 kernel, 避免反复经过全局显存。

视频证据时间窗: 00:32:55–00:33:35。

融合的真正对象是数据移动

“kernel 数减少”只是表象。若融合导致寄存器压力过高、占用率下降或重复计算暴涨, 端到端可能变慢。评估时应比较总 HBM bytes、持续时间和占用率, 而非只看 launch 数。

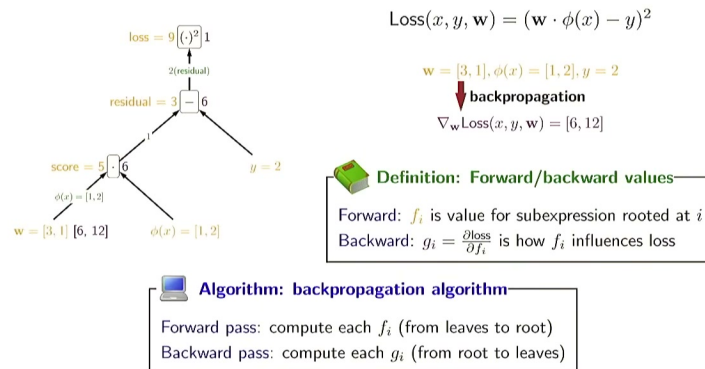
5.2 重计算: 用便宜 FLOPs 换昂贵访存

反向传播需要前向中间量。若某个中间量计算便宜但保存和读取昂贵, 可在反向时重算。令保存方案的额外访存成本为 T_{save} , 重算成本为 $T_{\text{recompute}}$, 当

$$T_{\text{recompute}} < T_{\text{save}}$$

- $T_{\text{recompute}}$: 重新执行局部算子的时间;
- T_{save} : 写出并在反向读回中间激活的时间;
- 条件成立时, 重计算既省显存又可能更快。

Trick 3: recomputation



[From cs221]

In backpropagation, we store the activations (yellow) and compute Jacobians (green)

图 18: 课程以激活函数链说明重计算可省去多次显存访问。

视频证据时间窗: 00:37:10-00:37:50。

重计算不是免费的

矩阵乘通常昂贵, 不宜盲目重算; 随机算子还必须重现 RNG 状态。最佳检查点位置取决于每层激活大小、计算强度和带宽, 而不是固定“每几层存一次”。

5.3 本章小结

融合减少中间张量的落地, 重计算彻底放弃部分中间张量。它们的共同逻辑是: 当全局内存往返比局部计算更贵时, 用计算换数据移动。

6 合并访问、分块与对齐

6.1 让相邻线程访问相邻地址

DRAM 以突发事务搬运连续地址。若一个 warp 的线程访问连续且对齐的数据, 少量事务即可服务全部线程; 若地址跨越许多 cache line, 硬件要发出更多事务。可用请求字节与实际传输字节之比描述效率:

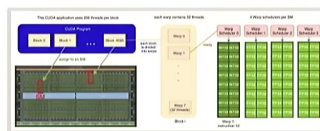
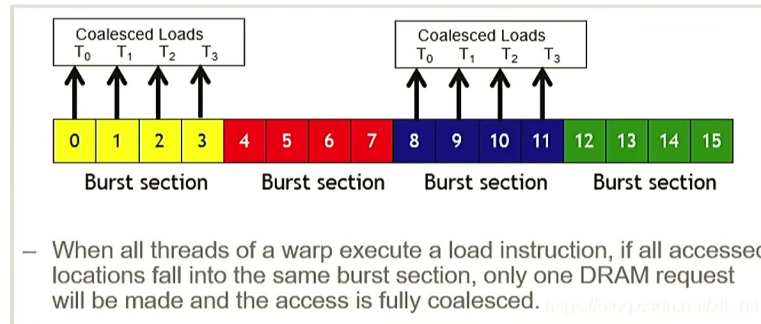
$$\eta_{\text{mem}} = \frac{Q_{\text{requested}}}{Q_{\text{transferred}}}$$

- $Q_{\text{requested}}$: 程序真正需要的数据量;

- $Q_{\text{transferred}}$: 内存系统实际搬运的数据量;
- 合并且对齐时该比值接近 1。

Memory coalescing

Memory accesses are *coalesced* if all the threads (in a warp) fall within the same burst



Reminder: a warp is a set of 32 consecutively numbered threads that execute together in a block. Memory accesses happen together

图 19: 相邻线程访问连续地址可合并成少量 DRAM 事务。

视频证据时间窗: 00:42:55–00:43:35。

先决定线程映射，再写索引

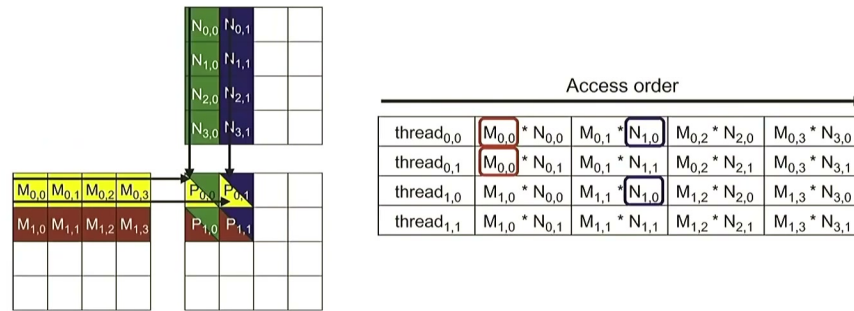
矩阵在内存中通常按行连续。让 `threadIdx.x` 变化对应连续列，最容易获得合并访问。若线程索引沿不连续维度走，即使数学上正确，也会造成低带宽利用率。

6.2 分块提升复用

Trick 5 (the big one): tiling

Tiling is the idea of grouping and ordering threads to minimize global memory access.

Let's go back to matrix multiplication..



Note that memory access is not coalesced, and repeated (M_{0,0} and N_{1,0})

图 20: 将矩阵切成 tile, 并把复用数据暂存 shared memory。

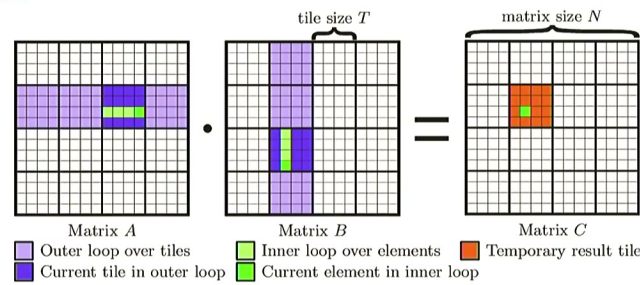
视频证据时间窗: 00:46:40-00:47:20。

朴素矩阵乘中, 一个输入元素可能从 HBM 重读 N 次。若 tile 边长为 T , 理想情况下全局读取次数降为约

$$R_{\text{tilled}} \approx \frac{N}{T}.$$

- N : 参与累加的共享维度长度;
- T : 片上 tile 覆盖的共享维度;
- 带宽流量理想减少约 T 倍, 但需付出同步和边界处理。

Tiling math



Non-tiled matrix multiply: each input is read N times from global memory

Tiled matrix multiply: each input is read $\frac{N}{T}$ times from global memory, and T times within each tile. This is a factor of T reduction in global memory access



图 21: 分块把输入从全局内存的重复读取由 N 次降到约 N/T 次。

视频证据时间窗: 00:50:55–00:51:35。

若每块使用共享内存 S_{block} , 每个 SM 可用容量为 S_{SM} , 则仅由共享内存给出的驻留块上限为

$$B_{\text{resident}} \leq \left\lfloor \frac{S_{\text{SM}}}{S_{\text{block}}} \right\rfloor.$$

- S_{block} : 输入 tile、输出暂存和 padding 的总量;
- S_{SM} : 一个 SM 可分配的 shared memory;
- tile 过大可提高复用, 却可能降低并发块数。

tile 大小是一组联合约束

它必须适应矩阵形状、Tensor Core 指令粒度、共享内存、寄存器和 block 线程数。边界不整除时还要 mask; 数据未对齐时要承担额外事务。最优值通常需要针对代表性形状实测。

6.3 本章小结

合并访问提高一次事务的有效载荷, 分块提高一次载入的复用次数。两者一起决定真实 Q , 也是把 kernel 从 Roofline 带宽斜线向计算平台推进的主要手段。

7 调度粒度与波量化

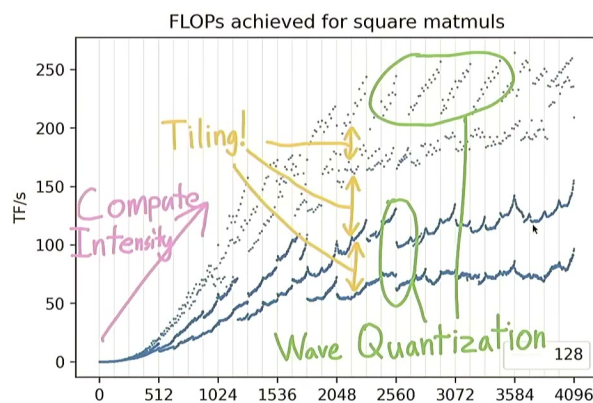
7.1 矩阵只大一个元素为何可能突然变慢

线程块不能被拆到多个 SM 上。假设网格有 G 个块、设备有 M 个 SM，每个 SM 每一波执行一个块，则波数近似

$$N_{\text{waves}} = \left\lceil \frac{G}{M} \right\rceil.$$

- G : 由输出形状和 tile 尺寸决定的 block 数;
- M : 可并行接收 block 的 SM 数;
- G 刚越过 M 的整数倍时，会新增一整波尾部工作。

Matrix mystery



We understand some of this (compute intensity, tiling). Let's take a closer look.

🔍 🔍 🔍 🔍 🔍 🔍 🔍 🔍 🔍 🔍

图 22: 矩阵尺寸从 1792 到 1793 时，tile 数和尾波结构突变。

视频证据时间窗：00:59:10–00:59:50。

性能曲线不是连续函数

形状的一个元素变化，可能改变 tile 数、对齐、Tensor Core 适配或编译特化路径。基准测试应扫描真实形状分布，而不是只测整齐的二次幂。

若最后一波只有 $r = G \bmod M$ 个块，其 SM 利用率近似

$$u_{\text{tail}} = \frac{r}{M} \quad (r \neq 0).$$

- r : 尾波实际有工作的 SM 数;
- $M - r$: 尾波空闲 SM 数;
- 增加 block 并行度或调整 tile 可减轻量化损失。

基准测试要记录形状

只报告“某 kernel 快 20%”没有迁移价值。必须记录批量、序列长度、隐藏维度、精度、布局和 GPU 型号，并给出预热、同步和重复统计；否则测到的可能只是编译、缓存或尾波偶然性。

7.2 本章小结

调度离散性解释了许多看似反常的性能跳变。Roofline 描述连续上界，波量化、占用率和形状特化决定能否接近这个上界。

8 FlashAttention: 把原则组合成算法

8.1 普通注意力的中间矩阵

缩放点积注意力为

$$S = \frac{QK^\top}{\sqrt{d}}, \quad P = \text{softmax}(S), \quad O = PV.$$

- $Q, K, V \in \mathbb{R}^{n \times d}$: 查询、键和值;
- $S, P \in \mathbb{R}^{n \times n}$: 得分与概率矩阵;
- $O \in \mathbb{R}^{n \times d}$: 输出;
- 朴素实现把 S 、 P 写入 HBM, 产生 $O(n^2)$ 中间流量。

Recap of part 2: making ML workloads go fast

❖ Reduce memory accesses

- ❖ Coalescing
- ❖ Fusion

❖ Move memory to shared memory

- ❖ Tiling

❖ Trade memory for compute/accuracy

- ❖ Quantization
- ❖ Recomputation

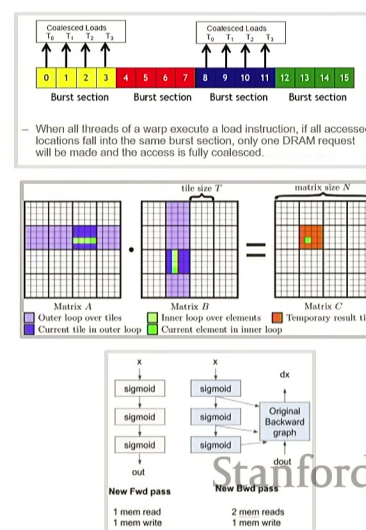


图 23: 注意力由 $QK^\top$ 、softmax 和 PV 三部分构成。

视频证据时间窗: 01:03:40–01:04:20。

8.2 在线 softmax 保持数值稳定

对一行 logits, 稳定 softmax 使用

$$p_i = \frac{\exp(x_i - m)}{\sum_j \exp(x_j - m)}, \quad m = \max_j x_j.$$

- m : 该行最大值, 用来避免指数溢出;
- 分母: 归一化常数;
- 分块处理时, m 和分母必须可增量合并。

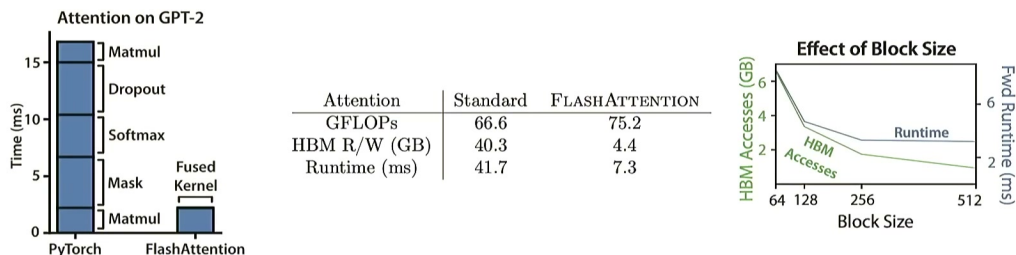
处理新块最大值 m_b 与指数和 ℓ_b 后, 在线状态更新为

$$m' = \max(m, m_b), \quad \ell' = e^{m-m'} \ell + e^{m_b-m'} \ell_b.$$

- (m, ℓ) : 已处理块的行最大值和指数和;
- (m_b, ℓ_b) : 当前块统计量;
- 缩放因子把不同最大值基准下的部分和转换到同一基准。

Part 3: Using what we know to understand Flash Attention

Flash attention [Dao et al] dramatically accelerates attention.. But how?



Technique from paper:

We apply two established techniques (tiling, recomputation) to overcome the technical challenge of computing exact attention in sub-quadratic HBM accesses. We describe this in Algorithm 1. The main idea



图 24: 在线 softmax 使一行得分可以按块流式归约。

视频证据时间窗: 01:05:25-01:06:05。

算法创新来自 I/O 复杂度

FlashAttention 并未改变注意力的数学结果。它改变了计算顺序: 把 Q, K, V 分块搬入片上存储, 在线维护归一化状态, 避免完整 $n \times n$ 矩阵往返 HBM。收益来自融合、分块和重计算的组合。

若片上 tile 为 $B_r \times B_c$, 每次只保留局部得分块, 则额外得分存储从

$$M_{\text{naive}} = O(n^2)$$

- n : 序列长度;
- 朴素实现显式保存全部注意力得分或概率;
- 长上下文时, 该项很快超过参数之外的显存预算。

降到分块工作集

$$M_{\text{tile}} = O(B_r B_c + nd).$$

- B_r, B_c : 查询行块与键列块大小;
- nd : 输入输出张量的线性规模;
- 反向可重算局部概率, 继续避免保存完整 P 。

Tiling part 2: incremental computation of the softmax

From Mikailov and Gimelshein 2018,

Normal softmax

$$y_i = \frac{e^{x_i - \max_{k=1}^V x_k}}{\sum_{j=1}^V e^{x_j - \max_{k=1}^V x_k}} \quad (2)$$

All major DL frameworks are using this safe version for the Softmax computation: TensorFlow

Algorithm 2 Safe softmax

```

1:  $m_0 \leftarrow -\infty$ 
2: for  $k \leftarrow 1, V$  do
3:    $m_k \leftarrow \max(m_{k-1}, x_k)$ 
4: end for
5:  $d_0 \leftarrow 0$ 
6: for  $j \leftarrow 1, V$  do
7:    $d_j \leftarrow d_{j-1} + e^{x_j - m_V}$ 
8: end for
9: for  $i \leftarrow 1, V$  do
10:   $y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$ 
11: end for
```

Online softmax

Algorithm 3 Safe softmax with online normalizer calculation

```

1:  $m_0 \leftarrow -\infty$ 
2:  $d_0 \leftarrow 0$ 
3: for  $j \leftarrow 1, V$  do
4:    $m_j \leftarrow \max(m_{j-1}, x_j)$ 
5:    $d_j \leftarrow d_{j-1} \times e^{m_{j-1} - m_j} + e^{x_j - m_j}$ 
6: end for
7: for  $i \leftarrow 1, V$  do
8:    $y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$ 
9: end for
```

To keep track of the max, incrementally update the max, and set up a telescoping sum

This lets you compute the softmax tile-by-tile

图 25: FlashAttention 前向按 Q 块和 K, V 块遍历并在线更新输出。

视频证据时间窗: 01:09:25–01:10:05。

迁移到其他 kernel 的方法

先写数学等价式, 再列出必须跨 HBM 保存的状态; 寻找可结合的归约统计量; 按片上容量分块; 最后检查对齐、尾波、精度和反向重计算。这比从 CUDA 语法开始更容易得到正确的高性能设计。

技巧	主要减少什么	典型收益	主要风险
低精度	每元素字节和计算成本	带宽、Tensor Core	溢出与收敛
融合	中间张量读写与 launch	逐元素链、归约	寄存器压力
重计算	保存的激活	显存、HBM 流量	额外 FLOPs
合并访问	无效内存事务	有效带宽	布局约束
分块	重复全局读取	算术强度	资源占用、边界
在线归约	全量中间矩阵	渐近 I/O 复杂度	数值状态设计

Recap for the whole lecture

- ❖ Hardware powers scale, and low-level details determine what scales or doesn't
- ❖ Current GPU based compute strongly encourages thinking about matmul + data movement
- ❖ Thinking carefully about the GPU (coalescing, tiling, fusion) leads us to good performance

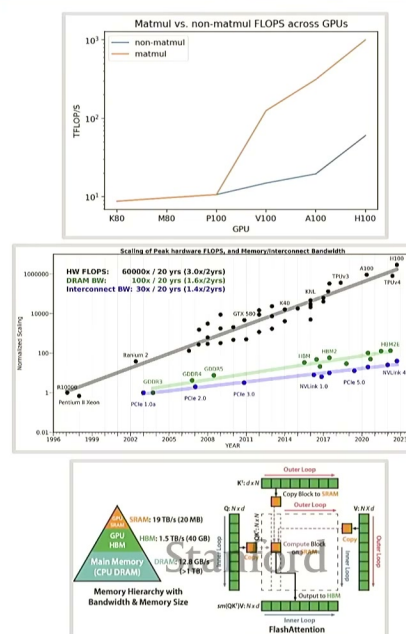


图 26: 从 GPU 硬件到 FlashAttention 的完整性能推理链。

视频证据时间窗: 01:12:40-01:13:20。

最终检查清单

遇到慢 kernel, 依次问: 它是计算受限还是带宽受限? 真实 DRAM bytes 是多少? 线程访问是否合并? 是否存在中间张量落地? tile 是否提高复用却压低占用? 形状是否产生尾波? 最后才问要不要手写 CUDA。

8.3 本章小结

FlashAttention 是本讲原则的综合实例: 用分块提高复用, 用在线 softmax 保持数学等价, 用融合消除中间写回, 用重计算换显存。真正可迁移的知识不是一个 kernel 名字, 而是以 I/O、算术强度和调度粒度为中心的推理方法。

9 性能诊断推演与复习

9.1 从症状建立假设

面对一个陌生的慢算子，第一步不是改代码，而是描述症状：输入形状是否固定，时间是否随元素数线性增长，精度改变是否明显加速，批量增大时吞吐是否趋于平台，某些边界形状是否突然退化。线性增长且低吞吐通常指向带宽；大矩阵吞吐接近峰值可能已经计算受限；短算子对批量很敏感往往是 launch 主导；只有少数形状异常则优先检查 tile、对齐和尾波。

第二步写出可反证预测。若怀疑带宽受限，融合两个阶段应减少 DRAM bytes 并近比例降低时间；若怀疑计算受限，换低精度或 Tensor Core 布局应显著加速；若怀疑分歧，按条件重新分组数据应改善 warp 执行效率；若怀疑尾波，轻微改变 tile 数应出现阶梯状变化。一个解释若不能预测下一次实验，就只是故事。

最小证据包

保存硬件型号与功耗状态、软件版本、完整输入 shape/stride/dtype、预热和重复协议、kernel 时间线、DRAM bytes、有效带宽、计算吞吐、网格与 block 配置。这样即使优化失败，也能留下可复查的系统知识。

9.2 建立形状矩阵

语言模型 kernel 不能只测一个尺寸。序列长度会改变注意力得分面积，microbatch 改变矩阵的 m 维，隐藏维度决定 Tensor Core 对齐，词表和专家数还会产生不整除边界。实践中应从训练日志抽取高频形状，按出现概率加权；再加入刚好跨过 tile、warp 或波次边界的“对抗形状”，最后保留极小和极大输入检查固定开销与容量极限。

对每个形状同时记录中位数、较慢分位数和端到端占比。一个 kernel 单次提速很大，如果只占 step 的很小部分，整体收益有限；相反，一个只快百分之几却覆盖每层、每步的操作，可能更值得维护。优化排序应使用“可节省的总训练时间”，而不是最醒目的局部加速比。

把正确性条件放进基准

每个性能样本先与参考实现比较输出和梯度，覆盖 NaN、极值、非整除形状和不同精度。否则最快结果可能来自漏算边界、错误广播、溢出或悄悄降低精度。性能回归和数值回归应由同一组 shape 驱动。

9.3 优化阶梯

一个稳健的阶梯是：先消除不必要的 Python/主机同步；再使用框架已有 fused 算子或编译器；随后改变布局、精度和批量以进入高效库路径；只有 profiler 证明仍存在大额暴露时间时，才写 Triton 或 CUDA。进入自定义 kernel 后，依次检查访问合并、全局流量、片上复用、寄存器与共享内存、网格规模和尾波，而不是同时改动所有因素。

每次只改变一个主要变量并记录预测。若扩大 tile，预期全局读取下降但寄存器和共享内存上升；若增加 warp，预期并行归约加快但单 program 资源更高；若融合更多阶段，预期 HBM 流量下降但

代码尺寸和寄存器存活期增长。结果与预测相反时，应回到 profiler 更新模型，而不是继续“调参数直到快”。

9.4 FlashAttention 的迁移练习

可以把相同思路用于交叉熵、归一化、分块稀疏算子和 MoE 路由。先找出会被完整写回的中间张量，再判断是否存在可结合的在线统计量。例如归一化需要计数、和与平方和；log-sum-exp 需要最大值与缩放后的指数和；分块 Top- k 需要维护候选集合。只有统计量能在块间正确合并，流式算法才既省 I/O 又保持语义。

反向传播还要问：保存中间量还是重算？答案取决于中间量大小、重算强度和数值状态。重算 softmax 概率往往划算，重算大型矩阵乘未必划算；保存随机掩码也许比保存全部激活便宜。真正的设计对象是前向与反向的联合时间—显存曲线。

常见错误归因

“GPU 没跑满”不是根因；“显存还有空”不等于 tile 应继续加大；“kernel 少了”不等于流量少了；“occupancy 低”也不自动意味着提高它会更快。所有结论都要回到可测量的字节、FLOPs、依赖和资源限制。

9.5 复习问题

复习时应能独立回答：为什么峰值 FLOPs 不等于训练吞吐；如何从 dtype 和读写次数算逐元素强度；为何融合、重计算和分块本质上都在改变 I/O；warp 分歧与线程块尾波有何区别；tile 变大为何同时提高复用又可能降低占用；在线 softmax 如何合并不同块的最大值与指数和；以及为什么 FlashAttention 改变的是内存访问复杂度而不是注意力数学。

9.6 本章小结

性能工程是一条证据链：症状形成假设，模型给出预测，基准和 profiler 检验预测，正确性测试保护语义，形状矩阵限定结论边界。掌握这条链，就能把本讲的 GPU 原理迁移到新的模型、算子和硬件，而不会依赖某个固定配方。